

Fuzz Testing

Perform Testing Procedure

Revision

Version 1
6/18/26 8:05 PM

Author

Charles Wilson

Abstract

This document describes the procedure used to undertake the **perform testing** activity described in the **AVCDL** ^[1] **Fuzz Testing Report** ^[2] secondary document.

Group / Owner

Security / Security Architect

Motivation

This document is motivated by the need to have runtime-specific, cybersecurity-related feedback in the development of software for use within safety-critical, cyber-physical systems.

Note: Within the context of this document, the terms ***security*** and ***cybersecurity*** are used interchangeably. It is presumed that the term ***security*** is being used in reference to ***cybersecurity*** and not ***physical security***.

License

This work was created by **Torc Robotics** and is licensed under the **Creative Commons Attribution-Share Alike (CC BY-SA-4.0)** License.

<https://creativecommons.org/licenses/by/4.0/legalcode>

Audience

The audience of this document is the cybersecurity practitioner who will be conducting cybersecurity-specific fuzz testing.

Completeness of Output

Since fuzz testing is an as-is analysis, it is critical to ensure that the information gathered is as accurate and complete as possible. As with any cybersecurity assessment, it is not the place of the fuzz testing SME to make assumptions on the part of engineering. All information should be sourced from and confirmed by the owner of the element under consideration.

Disposition of Output

This activity does not produce documentation as an output.

Entry Criteria

This document assumes that the reader understands the purpose of cybersecurity-specific fuzz testing. Further, that the reader has read and understood the following:

- **Fuzz Testing Report** (AVCDL secondary document)
- **Fuzz Testing – Establish Settings Procedure** (AVCDL tertiary document)
- **Fuzz Testing – Create Test Procedure** (AVCDL tertiary document)

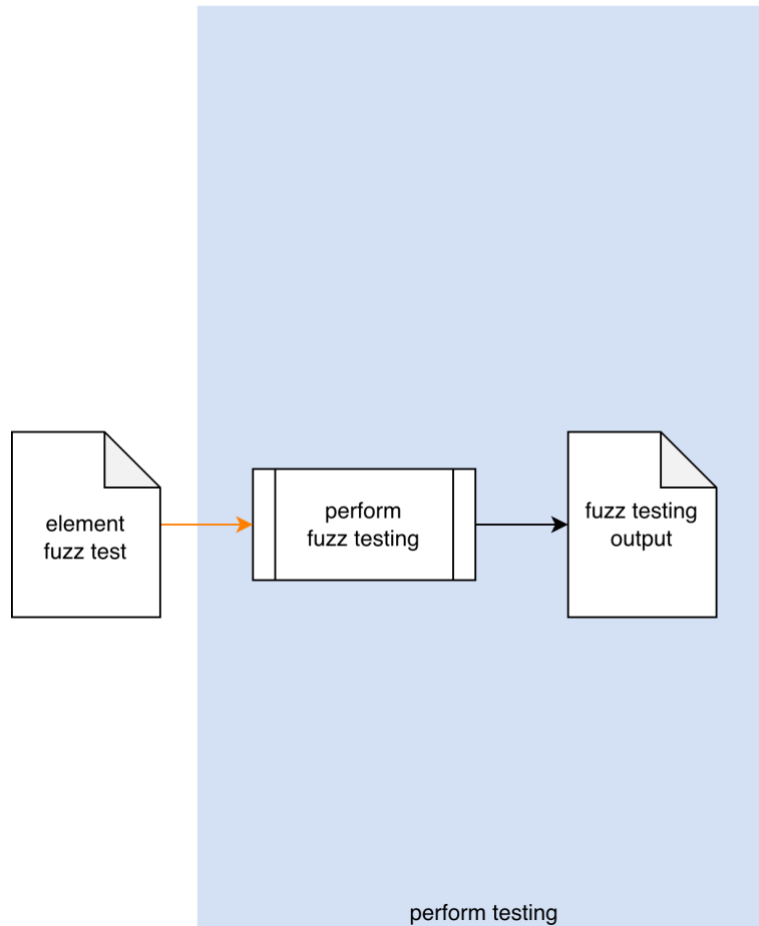
Prerequisites – Input Materials

It is required that the fuzz test created in the **create test** activity for the element under consideration be integrated into the CI/CD system and available for use.

Note: If dynamic analysis is used in conjunction with fuzz testing, it is required that the appropriate modifications to the CI/CD system have been made to enable its use.

Perform Testing Activity

The workflow diagram for the **perform testing** activity of the **Fuzz Testing** process is shown below.



The build system invokes the **Element Fuzz Test**. The generated output is captured into a **Fuzz Testing Output** file.

Note: As shown above, this activity is presumed to be performed within the CI/CD pipeline, and as such requires no participation by people.

Note: Where possible, it is recommended that the fuzz testing output be a SARIF ^[6] encoded JSON file.

Methodology

If the preceding activities of **settings establishment** and **test creation** have been properly followed, this activity should just work. Its inclusion should therefore be simply to provide completeness to the fuzz testing process. Any issues which arise should fall into one of the categories covered in the next section.

Considerations

The following items need to be taken into consideration when performing fuzz testing. Some of these should have been addressed during the **establish settings** ^[3] and **create test** ^[4] activities. We come back to them because although they should have been specifically addressed during the **Secure Settings - Apply Options** ^[5] activity and the pre-flight of the test within the CI/CD system during the **create test** activity, they may still need to be addressed within the context of the live CI/CD process.

Note: **Footprint** is not discussed here because it has been addressed at length in earlier procedures.

Note: **Concurrency** is not discussed here because it has been addressed in the **Fuzz Testing – Create Test Procedure** document.

Termination on Error

Since fuzz testing may induce program termination, important to adjust the test suite to take this into consideration.

Issue Occlusion

Discovered issues may mask more fundamental ones. Be sure to adjust the test to verify that the issue discovered is the root issue.

Cybersecurity Side-effects

Because fuzz testing requires intrusive access to data structures within the code, there may be a significant impact on other cybersecurity-related tooling. It is highly recommended that fuzz testing be conducted in isolation from other cybersecurity tooling (with the exception of dynamic analysis) in order to not trigger false positives in those tools.

Note: This is especially true when fuzz testing is being augmented with dynamic analysis.

Exit Criteria

This procedure is considered complete once the **Fuzz Testing Output** has been generated.

References

1. **AVCDL** (AVCDL primary document)
2. **Fuzz Testing Report** (AVCDL secondary document)
3. **Fuzz Testing - Establish Settings Procedure** (AVCDL tertiary document)
4. **Fuzz Testing - Create Test Procedure** (AVCDL tertiary document)
5. **Secure Settings - Apply Options Procedure** (AVCDL tertiary document)
6. **Static Analysis Results Interchange Format (SARIF) Version 2.1.0**
<https://docs.oasis-open.org/sarif/sarif/v2.1.0/os/sarif-v2.1.0-os.pdf>